

A3 LLM 訓練資料的多層級 Dedup

一、前言

(一)研究背景與動機

大型語言模型的訓練高度仰賴高品質語料庫。然而，網路語料中普遍存在逐字複製或改寫後的重複資料，這不僅會徒增運算成本，更會削弱模型的泛化能力。

目前主流的字面去重方法雖然高效，但難以辨識經過改寫的語意重複。為解決此問題，近年研究提出了基於高維向量的語意去重技術。然而，其依賴的 K-Means 分群流程在面對海量資料時，計算與記憶體負擔極大，且硬性分群容易導致語意相近的文件被錯開，產生跨群集遺漏問題。

為突破此效能瓶頸，本研究引入近似最近鄰搜尋中的 HNSW 演算法取代 K-Means，透過多層圖結構將搜尋複雜度降至接近 $O(\log N)$ ，有效降低運算成本與遺漏率，建立一套兼顧效率與準確率的語意去重流程。期望能降低計算成本，同時逼近 SemDeDup 原本的去重效果。

(二)研究問題

為了建立高效率且具實務可行性的語意去重架構，本研究聚焦於以下幾個問題。

RQ1：大型網路語料中的語意相似度分布具有何種特性？

現有研究普遍認為大型語料集中存在大量語意重複內容，但在經過 MinHash 等字面層級去重處理後，文件間的語意相似度分布仍缺乏系統性的量化分析。

因此，本研究首先分析 SlimPajama 資料集中的文件相似度分布，觀察 Cosine Similarity 的實際分佈情形，藉此了解語意重複內容的比例與特性，並作為後續候選文件搜尋機制設計的依據。

RQ2：低成本向量表示是否足以作為語意去重的候選文件產生器？

語意去重的主要成本來自高品質 Embedding 的產生與後續檢索過程。因此，一個自然的問題是：是否能利用計算成本較低的 Static Embedding 取代 Transformer Embedding，作為語意去重流程中的第一階段候選文件產生器 (Candidate Generator)。

為回答此問題，本研究比較 Static Embedding 與 Transformer Embedding 所產生的 Top-k Nearest Neighbors 一致性，分析兩者在不同相似度區間下的近鄰結構差異，並評估低成本向量表示是否能可靠保留真正的語意鄰居。

RQ3：HNSW 是否能有效取代 SemDeDup 中基於 K-Means 的候選文件產生流程？

RQ2 探討了低成本 Embedding 作為候選文件產生器的可行性。然而，若單純依賴低成本向量表示無法維持足夠的候選召回率，則需要尋找其他更有效率的候選搜尋機制。

因此，本研究進一步探討是否能利用 HNSW 近似最近鄰搜尋直接建立候選文件池，以取代傳統 SemDeDup 中基於 K-Means 的候選文件產生流程。透過評估不同 Embedding 模型與 HNSW 配置下的 Recall、QPS 以及建表時間，分析 HNSW 作為候選文件產生器的可行性與效益。

RQ4：各項系統優化機制對去重品質與執行效率的貢獻為何？

除了驗證 HNSW 的有效性之外，本研究亦希望量化不同優化機制對整體系統效能的影響。

因此，我們建立多種系統版本，透過逐步替換候選文件產生機制與優化 HNSW 配置，量測各版本在執行時間、QPS、記憶體使用量以及去重品質上的差異，以分析各項優化措施的實際貢獻。

二、研究架構

為回答上述研究問題，我們設計了以下幾組實驗。

實驗一：語意相似度分布分析

1. 實驗目的：

分析 SlimPajama 資料集中的語意相似度分布特性，探討 MinHash 去重後殘留語意重複內容的比例與分布情形。

2. 量測方式：

- 從 SlimPajama 或 C4 取一個小 subset (例如 100K 筆文件)。
- 用 transformer embedding 把每篇文件轉成向量。
- 對每篇文件找 k-nearest neighbors，記錄 cosine similarity。
- 畫出 similarity 的 histogram。

3. 問題討論：

- (1) 分布是什麼形狀？大多數文件對的相似度集中在哪裡？
- (2) 「模糊地帶」(例如 cosine 0.7 到 0.95) 大概佔了多少比例？

實驗二：比較不同 embedding 的 nearest neighbor 一致性

1. 實驗目的：

本實驗比較 Static Embedding 與 Transformer Embedding 所建立的近鄰結構 (Neighborhood Structure)，分析兩者在不同相似度區間中的一致性。

2. 量測方式：

使用與實驗一同一批文件，分別用 static embedding (Model2Vec) 和 transformer embedding 算 nearest neighbors。比較兩者找到的 top-k neighbors 有多少重疊。

3. 問題討論：

- (1) 在 cosine similarity 很高 (> 0.95) 或很低 (< 0.7) 的區間，兩種 embedding 的 neighbors 是否幾乎一致？
- (2) 在中間的模糊地帶，一致性下降多少？
- (3) 這個一致性的落差，對「用便宜方法先篩，再用貴的方法驗證」的策略有什麼影響？

實驗三：Embedding 模型與 HNSW 配置效能分析

1. 實驗目的：

本實驗探討 HNSW (Hierarchical Navigable Small World) 是否能作為 SemDeDup 中 Candidate Generation 階段的替代方案，並評估不同 Embedding 模型與 HNSW 配置下的去重品質與系統效能。

本實驗目標包含：

- 驗證 HNSW 作為 Candidate Generator 的可行性
- 分析不同 Embedding 模型對語意去重效果的影響
- 探討 Recall 與系統效能之間的權衡關係
- 選擇後續實驗所使用的最佳模型與配置

2. 量測方式：

建立 HNSW-based Semantic Deduplication Pipeline，並分別使用以下 Embedding 模型建立文件向量：

- intfloat/e5-small-v2
- intfloat/e5-base-v2
- BAAI/bge-small-en-v1.5
- BAAI/bge-base-en-v1.5
- sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
- sentence-transformers/all-MiniLM-L6-v2

對每個模型分別建立 HNSW 向量索引，並測試不同參數組合：

- HNSW Parameters
 - $M \in \{8, 16, 32\}$
 - $ef_construction \in \{100, 200, 400\}$
 - $ef_search \in \{50, 100, 200\}$
- Similarity Threshold
 - $0.80 \sim 0.98$

為觀察效能與準確率之間的權衡關係，本研究額外定義兩種代表性配置：

- Best-Recall Configuration：以最大化 Recall 為目標
- Fast Configuration：以最大化 QPS 為目標

量測指標包含：

- Candidate Recall
- Duplicate Recall
- Query Per Second (QPS)
- Index Build Time
- Memory Usage
- Final Deduplication Ratio

3. 問題討論：

- (1) 不同 Embedding 模型所形成的向量空間，對語意去重品質與候選召回率有何影響？
- (2) 在 HNSW 架構下，Embedding 模型規模與向量表示能力的提升，是否能轉換為實際的 Duplicate Detection 效益？
- (3) HNSW 關鍵參數 (M 、 $ef_construction$ 、 ef_search) 與 Similarity Threshold 對 Recall、QPS 以及建表時間的影響為何？
- (4) 在 Recall、QPS 與建表成本之間，是否存在具實務價值的最佳配置 (Sweet Spot) ？
- (5) 哪一種 Embedding 模型與 HNSW 配置最適合作為後續實驗與實際部署的預設方案？

實驗四：HNSW 參數敏感度分析 (Sensitivity Analysis)

1. 實驗目的：

HNSW 的搜尋品質與系統效能高度受到索引參數影響，不同配置可能導致 Recall、吞吐量以及資源消耗產生顯著差異。

因此，本實驗旨在分析 HNSW 關鍵參數對語意去重任務的影響，並探索不同 Embedding 模型下的最佳參數區間 (Sweet Spot)。此外，本研究亦觀察在不同 Similarity Threshold 下是否出現效能異常現象，例如 Recall 急遽下降、群集規模暴增或災難性塌陷 (Catastrophic Collapsing) 等情況。

2. 量測方式：

以實驗二中表現較佳之 Embedding 模型為基礎，建立 HNSW 向量索引，並針對(M , $ef_construction$, ef_search), Similarity threshold 參數進行敏感度分析。

3. 問題討論：

- (1) HNSW 關鍵參數 (M 、 $ef_construction$ 、 ef_search) 對 Candidate Recall、QPS 與建表時間的影響為何？
- (2) 是否存在 Performance Spike 或 Catastrophic Collapsing 等臨界現象？
- (3) 在 Recall、QPS 與資源消耗之間，是否存在具實務價值的最佳參數區間 (Sweet Spot) ？

實驗五：Pipeline 版本比較分析

1. 實驗目的：

驗證所提出架構是否能在維持去重品質的前提下，顯著降低整體計算成本與執行時間。

2. 量測方式：

建立四種不同系統版本

- V1：HNSW Structure：建立 HNSW 向量索引並執行近鄰搜尋，用於量測索引建構與候選搜尋本身的系統成本。
- V2：K-Means-based SemDeDup：採用原始 SemDeDup 流程作為基準。
- V3：HNSW and SemDeDup：將 HNSW 接在 SemDedup 之前，透過 HNSW 改善 Candidate Generation 階段的搜尋時間。
- V4：Optimized HNSW-based SemDeDup：進一步修改 SemDedup 流程以及演算法。

針對各個版本量測指標並且衡量每個版本對各個指標的貢獻度。

3. 問題討論：

- (1) HNSW 是否能有效取代 SemDeDup 中基於 K-Means 的 Candidate Generation 流程？
- (2) Candidate Generation 機制的替換對整體執行時間與去重品質帶來多少改善？
- (3) 不同優化階段 (V1 → V4) 對 Recall、QPS 與資源消耗的貢獻為何？
- (4) 在維持相近去重品質的前提下，所提出架構能達成多少效能提升？
- (5) 最終 Pipeline 是否能在大型語料語意去重任務中提供更具實務價值的效能 / 品質平衡？

三、實驗結果與討論

實驗資料庫與文獻背景

1. SlimPajama 資料集概述與預處理流程

本研究採用 SlimPajama (總計 627B Tokens) 作為實驗核心語料庫。該資料集係由 Cerebras 團隊針對原始 RedPajama (1.2T Tokens) 進行深度清洗與全域去重 (Global Deduplication) 後釋出的高質量文本集合。其原始預處理與去重管線 (Pipeline) 主要包含以下兩個階段：

(1) 基礎文本清洗：

- 移除所有標點符號、空白、換行與製表符，且直接剔除文本長度小於 200 個字元的極短文件 (此步驟共淘汰了 1.86% 的文件)。

(2) 字面層級去重：

- 採用 MinHash LSH 演算法作為去重核心。
- 配置參數：文本全小寫化、採用 13-grams 作為 Shingle 單位，並將 Jaccard 相似度閾值 (Threshold) 設定為 0.8。
- 此階段成功濾除高達 49.6% 的重複文件，大幅精簡了語料體積。

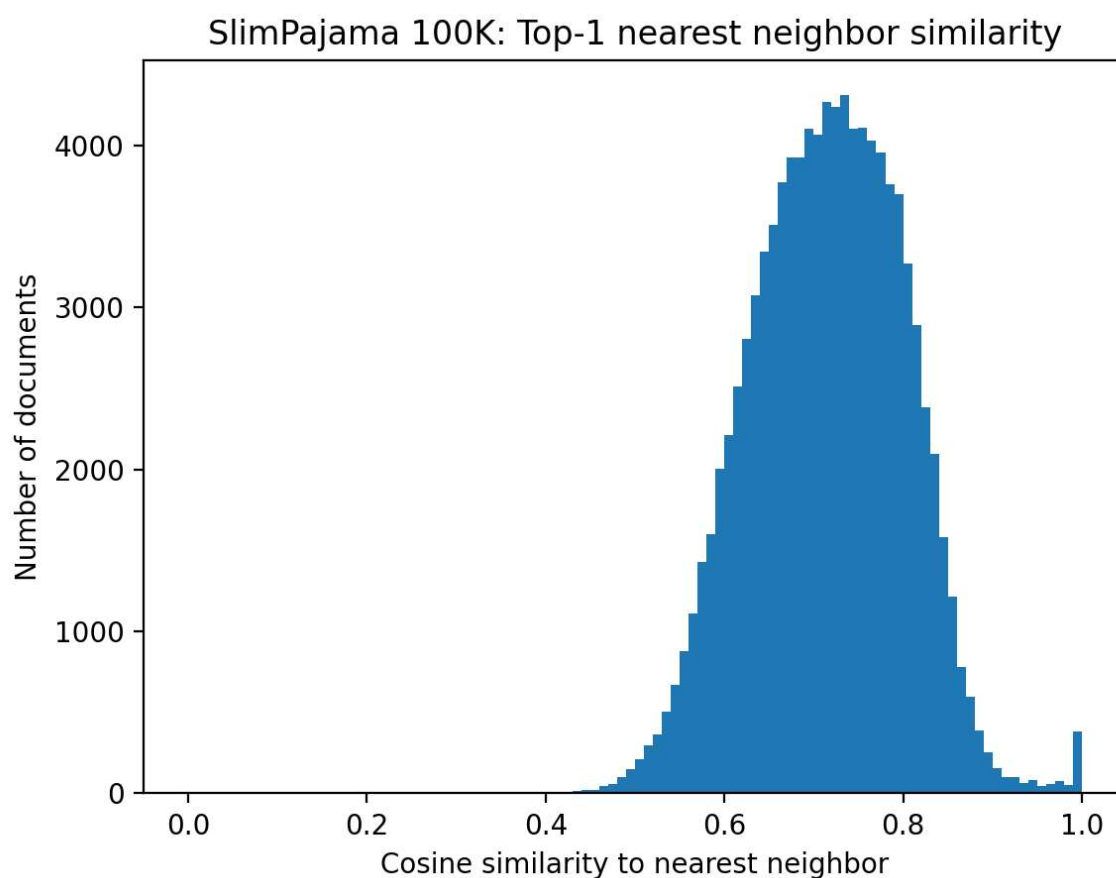
2. SlimPajama 數據分佈與各領域字面去重率綜合分析

相較於傳統大語言模型語料僅進行單一領域內 (Local) 的去重，SlimPajama 的核心優勢在於實現了跨領域的全域去重。以下為該資料集的組成結構，以及從 RedPajama 演進至 SlimPajama 過程中，各領域文本被 MinHash 剔除的比例 (重複率)：

bucket	num_docs	recall_Top10_in_StaticTop50_mean	miss_Top10_in_StaticTop50_mear
<0.50	434	0.113825	0.886175
0.50-0.70	42237	0.176563	0.823437
0.70-0.80	40526	0.301957	0.698043
0.80-0.90	15446	0.406442	0.593558
0.90-0.95	502	0.495020	0.504980
>=0.95	855	0.626433	0.373567

實驗一

本實驗自 SlimPajama 資料集中選取 100K 筆文本進行測試。下圖與下表分別展示了 Top-1 相似度之分布情形與統計數據。



Top-1 Similarity	Documents	Percentage
< 0.50	434	0.43%
0.50 - 0.70	42,237	42.24%
0.70 - 0.80	40,526	40.53%
0.80 - 0.90	15,446	15.45%
0.90 - 0.95	502	0.50%
>= 0.95	855	0.85%

我們可以看出，資料語意分布並未呈現明顯的雙峰分布，而是高度集中於中間區域。其中：

- Cosine Similarity 介於 0.50–0.70 的文件共 42,237 筆 (42.24%)
- Cosine Similarity 介於 0.70–0.80 的文件共 40,526 筆 (40.53%)
- Cosine Similarity 介於 0.80–0.90 的文件共 15,446 筆 (15.45%)

這三個區間合計達 98.22%，顯示絕大多數文件落在中高度相似範圍，而非集中於極端相似或極端不相似的兩端。

相反的，在高度相似 (Similarity > 0.95) 以及完全不相似 (Similarity < 0.50) 的文件比例非常

低，代表真正高度重複的文件比例很低，且完全不相似的文件亦相對少見。

此結果與一般 Filter-and-Verify 系統常見的雙峰假設不同，顯示經過 MinHash 預處理後，大部分殘留文件已不再屬於明顯重複案例，而是位於語意相近但難以直接判斷是否重複的區域。

```
ambiguous 0.50-0.7: 71.73%  
near duplicate >0.95: 0.14%
```

關於模糊地帶的比例，若將若將 Cosine Similarity 介於 0.70 至 0.95 定義為語意去重中的「模糊地帶」(Ambiguous Region)，則：

- 0.70–0.80：40,526 筆
- 0.80–0.90：15,446 筆
- 0.90–0.95：502 筆

總計為 56,474 筆，占整體資料集 56.47%。

若進一步將 0.50–0.70 區間納入考量，則有高達 98.22% 的文件落在 0.50–0.95 的中間區域。

此結果表示，MinHash 去除字面重複後，剩餘資料主要由大量語意相近但難以明確區分的文件所構成。換言之，真正需要語意去重系統處理的並非少數極端案例，而是大規模存在於中間相似度區間的模糊樣本。

因此，單純依賴低成本篩選器可能難以可靠區分這些文件，後續仍需引入更精細的候選搜尋與語意驗證機制，以維持足夠的召回率 (Recall) 與去重品質。

實驗二

實驗延續前述設定，選用 SlimPajama 100K 資料集進行評估，相關實驗數據統計請參閱下表。

bucket	num_docs	recall_Top10_in_StaticTop50_mean	miss_Top10_in_StaticTop50_mear
<0.50	434	0.113825	0.886175
0.50-0.70	42237	0.176563	0.823437
0.70-0.80	40526	0.301957	0.698043
0.80-0.90	15446	0.406442	0.593558
0.90-0.95	502	0.495020	0.504980
>=0.95	855	0.626433	0.373567

實驗結果顯示，Static Embedding (Model2Vec) 與 Transformer Embedding 所建立的近鄰結構並非高度一致。近鄰一致性會隨著 Cosine Similarity 降低而快速下降。在語意去重最重要的模糊區間 (Cosine Similarity 0.70–0.80) 中，Model2Vec Top-50 僅能覆蓋 Transformer Top-10 鄰居的 30.20%，代表約 69.80% 的真正語意鄰居被遺漏。

因此，即使在最容易辨識的高相似度區間，Static Embedding 仍無法完整重現 Transformer 所建立的鄰居關係；而在低相似度區間，兩者更呈現顯著差異。此結果說明 Model2Vec 對於部分明顯的 Duplicate 或 Near-Duplicate 文件具有一定辨識能力，但仍不足以直接取代 Transformer Embedding 作為語意去重的主要檢索依據。

為了實驗完整性，我們提高了 Candidate Pool 至 Top-100。

bucket	num_docs	recall_Top10_in_StaticTop50_mean	miss_Top10_in_StaticTop50_mear
<0.50	434	0.113825	0.886175
0.50-0.70	42237	0.176563	0.823437
0.70-0.80	40526	0.301957	0.698043
0.80-0.90	15446	0.406442	0.593558
0.90-0.95	502	0.495020	0.504980
>=0.95	855	0.626433	0.373567

經過調整後發現，即使將 Candidate Pool 擴增至 Top-100，Recall 也僅提升至 37.88%，仍有高達 62.12% 的 Transformer 鄰居無法被找回。此外，在 0.80–0.90 區間中：

- Top-50 Recall：40.64%
- Top-100 Recall：49.80%

即使增加一倍候選數量，仍無法達到足以支撐高品質 Candidate Generation 的召回率。

此結果顯示，Model2Vec 對於極端相似或極端不相似的文件尚具有一定區分能力，但對於大量存在於資料集中的中等相似度文件，其近鄰結構與 Transformer 存在明顯差異。

因此，單純以 Static Embedding 作為唯一 Candidate Source 的 Filter-and-Verify 架構並不安全，其召回率不足以支撐高品質的語意去重任務。且 Candidate Generation 的品質遠比 Verification 階段更為關鍵。即使後續驗證模型再精確，只要候選對未被送入驗證階段，就無法被最終發現。

本研究後續將進一步探討更有效率的候選搜尋機制，並提出以下幾項值得深入研究的方向：

- 探討多來源 Candidate Generation 機制，以降低單一向量表示所造成的候選偏差。
- 評估高召回率近似最近鄰搜尋方法，作為候選文件產生器的可行性。
- 探討不同相似度區間是否需要採用差異化的 Candidate Generation 策略，以改善模糊區間中的候選遺失問題。

實驗三

我們選擇了六種 Embedding 模型，並分別比較偏重效能之 Fast Configuration 與偏重搜尋品質之 High Recall Configuration，實驗結果表格如下

Model	Setting (M, efc, ef)	Recall	QPS	Build Time (s)	Query Time (s)	Candidate Pairs
intfloat/e5-base-v2	Fast (8,100,50)	0.93953	100131.40	0.18	0.20	2,952,010
	Good (32,400,200)	0.99985	9,995.27	1.28	2.00	3,072,815
intfloat/e5-small-v2	Fast (8,100,50)	0.87572	197089.17	0.71	0.10	4,374,437
	Good (32,400,200)	0.99888	15,775.27	2.85	1.27	4,334,114
all-MiniLM-L6-v2	Fast (8,400,50)	0.97565	352804.95	0.31	0.06	9,245
	Good (32,400,200)	0.99998	28,330.38	0.63	0.71	9,232
paraphrase-MiniLM-L12-v2	Fast (8,400,50)	0.97434	206947.79	0.87	0.10	9,899
	Good (32,400,200)	0.99999	15,916.70	1.22	1.26	9,899
bge-base-en-v1.5	Fast (8,100,50)	0.95610	84,391.16	0.52	0.45	18,898
	Good (32,400,200)	0.99990	7,412.42	3.47	3.72	18,911
bge-small-en-v1.5	Fast (16,100,50)	0.98730	211181.33	0.25	0.09	52,983
	Good (32,400,200)	0.99990	20,902.64	0.92	0.96	53,009

當 HNSW 參數由 Fast Configuration 調整至 High Recall Configuration 時，各模型之 Recall 均有提升。其中 Recall 平均提升約 4.8%，多數模型皆可接近或達到 99.9% 以上之 Candidate Recall。然而搜尋品質的提升伴隨顯著的系統成本增加。指標數值方面，各模型之 QPS 平均下降約 10.9 倍。索引建構時間平均增加約 4.8 倍，而查詢時間則平均增加約 9.6 倍。此結果顯示在大規模語意去重應用中，若系統對 Recall 並無極端要求，則採用 Fast Configuration 即可取得相當具競爭力之搜尋品質，同時保有較高之系統效能。

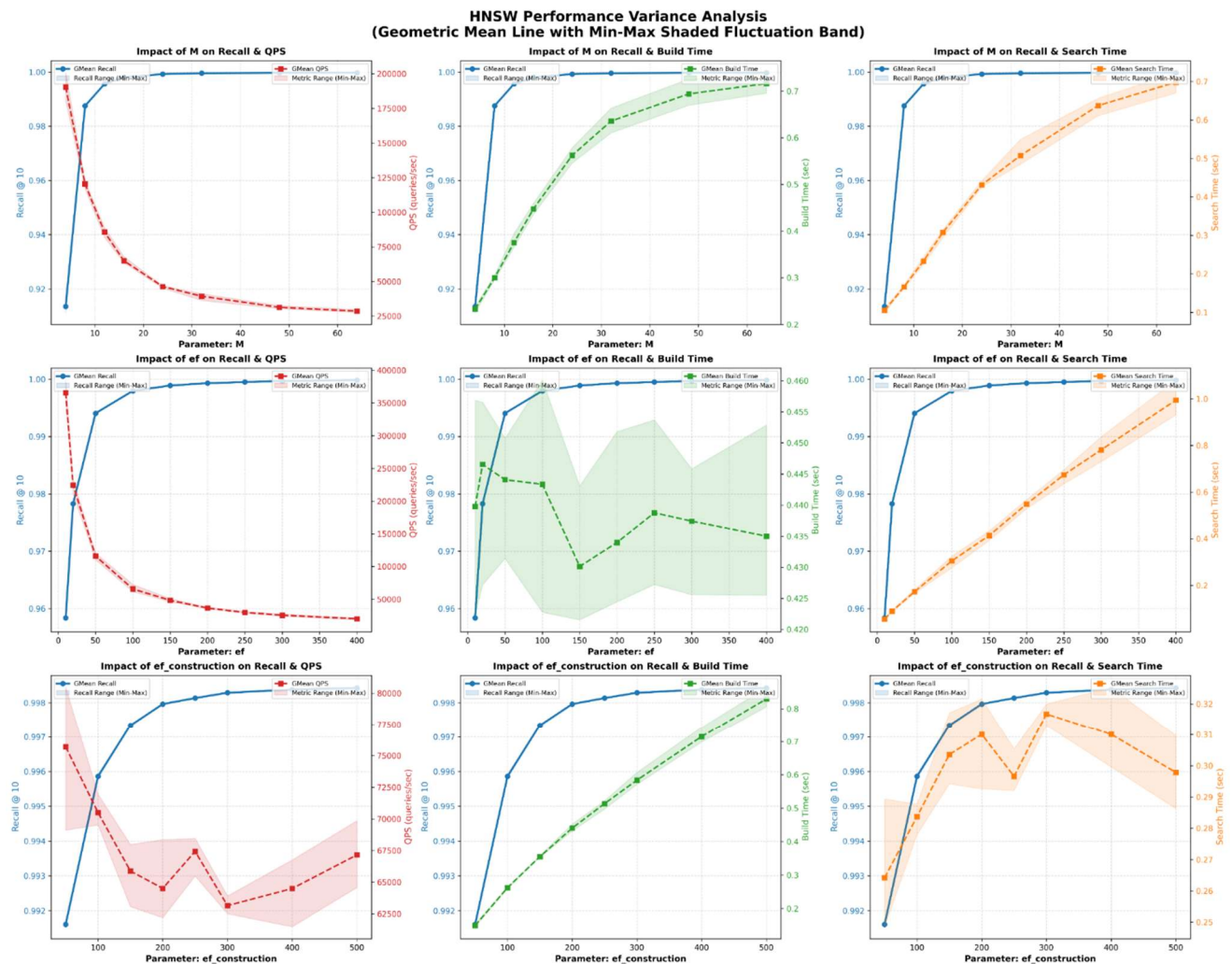
不同 Embedding 模型對 HNSW 參數之敏感度亦存在差異。其中 e5-base-v2 與 e5-small-v2 在不同參數組合下 Recall 波動較大，尤其 e5-small-v2 於 Fast Configuration 下僅達 87.57% Recall，顯示其搜尋品質較容易受到 HNSW 參數設定影響。相較之下，MiniLM 與 BGE 系列模型則展現較佳之穩定性，即使採用較保守之 HNSW 參數設定，仍能維持較高之 Candidate Recall。此外，本研究於 Similarity Threshold 實驗中亦觀察到，不同模型對 Threshold 變化之敏感程度有所不同。e5 系列模型在較低 Threshold 區間容易產生群集合併現象，顯示其向量空間中存在較明顯之語意連鎖效應；相對而言，MiniLM 系列模型之群集結構較穩定，即使在 Threshold =

0.80 的情況下仍未出現 Catastrophic Collapsing 現象。

綜合評估後，all-MiniLM-L6-v2 雖並非取得最高 Recall 的模型，但在 Recall、吞吐量、建表成本與群集穩定性之間展現最佳整體平衡，因此，本研究後續實驗將以 all-MiniLM-L6-v2 作為主要 Embedding 模型。

實驗四

實驗針對 M , ef , $ef_construct$ 對於 Recall, QPS, Search time, Build time 的 sensitivity analysis 實驗，結果如下表，陰影區域代表三次實驗的數值覆蓋範圍，最後數值則是取用幾何平均後的數值。



從表格可以發現，參數 M 對 Recall 的提升呈現顯著的對數增長趨勢。當 M 低於 8 時，由於圖結構的連通性不足，GMean Recall 處於 0.91 至 0.95 的低谷，但隨 M 增加，Recall 迅速突破 0.99 並趨於飽和。然而，這項精度的提升是以犧牲吞吐量為代價：圖的稠密度使每次查詢需遍歷的邊數大增，導致 QPS 從接近 200,000 篇/秒直線滑落至約 25,000 篇/秒，且建表時間與查詢時間皆隨 M 的增大呈現絕對的正線性增長。

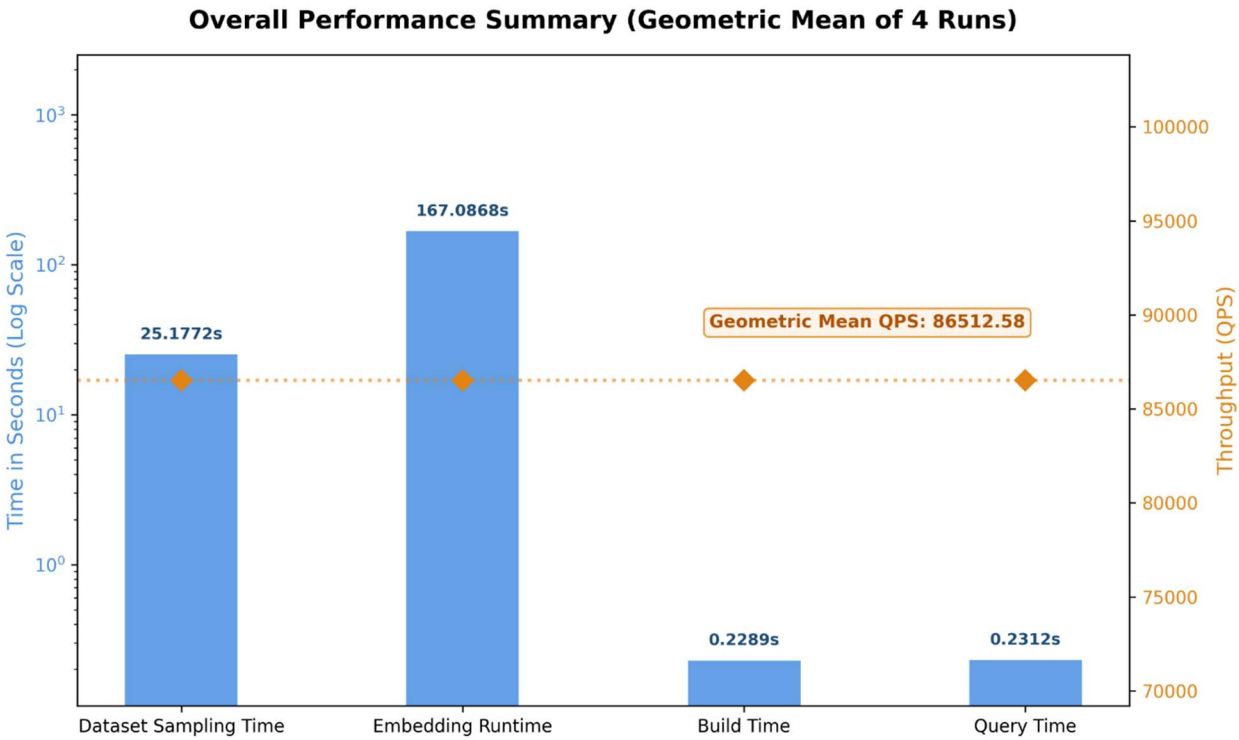
參數 ef 從 10 增加到 400 時，Recall 曲線在 $ef \leq 50$ 之前極為陡峭，隨後在 $ef \geq 100$ 時逼近 1.00 的完美召回率。然而，過大的搜尋範圍導致查詢延遲呈線性惡化（從 0.1 秒延長至 1.0 秒），引發 QPS 的斷崖式下滑。參數 $ef_construction$ 從 50 放大至 500，建表時間從 0.15 秒上升至 0.85 秒。此參數雖能拉升 Recall（從 0.9915 提升至 0.9985 以上），但對查詢延遲與 QPS 的直接影響較小，在整體趨勢中主要呈現非單調的波動。

實驗五

以下實驗若有使用 Embedding model 均使用 all-MiniLM-L6-v2，HNSW 參數均基準固定為(M, ef, ef_construct) = (16, 100, 100)，且所有實驗數據均實驗三次後取幾何平均。

V1 : HNSW Structure

Metric	Value
Dataset Sampling Time (s)	25.1772
Embedding Runtime (s)	167.0868
Build Time (s)	0.2289
Query Time (s)	0.2312
QPS	86512.5773
Recall@10	0.9959



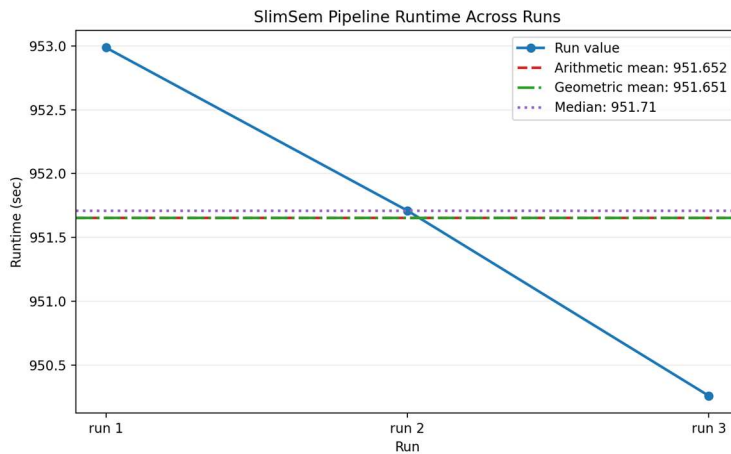
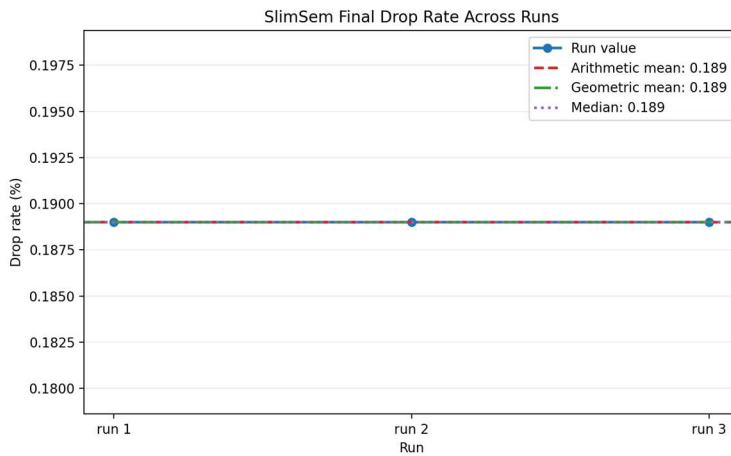
Script	Runtime (s)	Peak Memory (GB)
H01_sample_slimpajama.py	15.59	1.16
H02_embed.py	154.40	1.85
H03_hnsw.py	13.01	6.20
Total	183.00	9.20

V2 : K-Means-based SemDeDup

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 952.9867
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_sem/pipeline_20260614_161342.log
SUMMARY SAVED TO: /home/u08/workspace/HNSW/data/reports/pipeline_summary_slim_sem_20260614_161342.json
=====
```

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 951.7100
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_sem/pipeline_20260614_163728.log
SUMMARY SAVED TO: /home/u08/workspace/HNSW/data/reports/pipeline_summary_slim_sem_20260614_163728.json
=====
```

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 950.2592
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_sem/pipeline_20260614_165611.log
SUMMARY SAVED TO: /home/u08/workspace/HNSW/data/reports/pipeline_summary_slim_sem_20260614_165611.json
=====
```



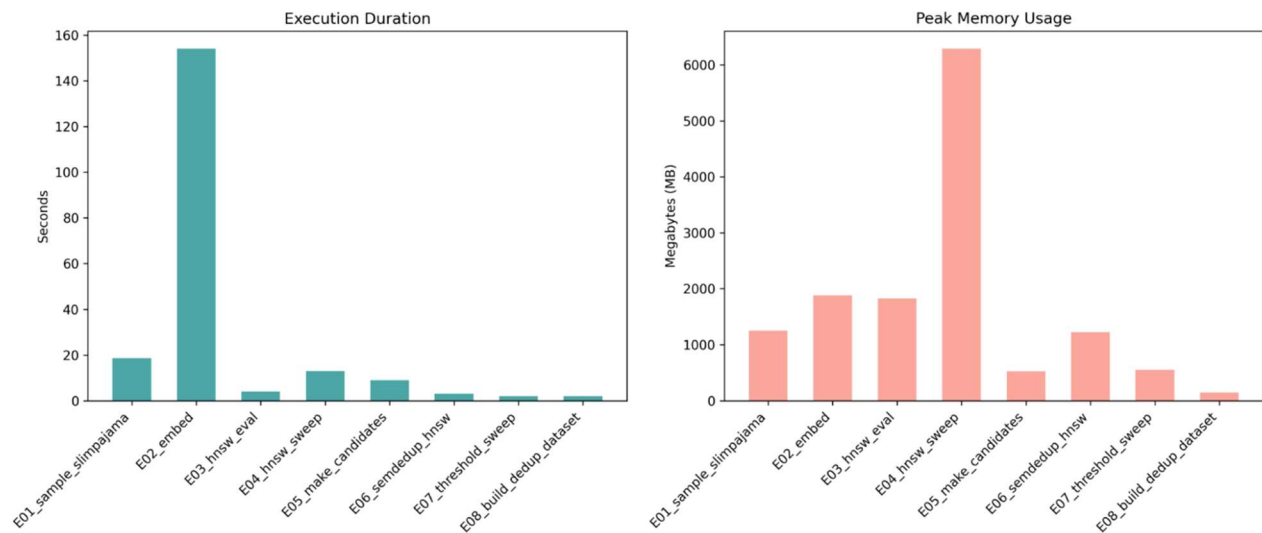
V3 : HNSW and SemDeDup

threshold	n_pairs	n_drop	drop_rate (%)	n_components	max_comp_size
0.8	7557	850	0.85%	99150	438
0.82	4779	679	0.679%	99321	403
0.84	2370	515	0.515%	99485	339
0.86	1040	362	0.362%	99638	232
0.88	515	195	0.195%	99805	56
0.9	266	98	0.098%	99902	27
0.92	106	55	0.055%	99945	21

根據 20,000 篇文件之基礎參數實驗結果，當相似度閾值設定在 0.88 附近時，最大連通元件規模有比較明顯的增益，且資料刪除率仍維持在合理區間。綜合評估去重效益與結構完整性，後續實驗以 0.88 與 0.90 作為關鍵閾值，且文件數提升到 100,000。

Threshold	Original Docs	Original Pairs	Pairs >= Threshold	Keep Docs	Drop Docs	Drop Rate
0.90	100,000	7,557	266	99,902	98	0.10%
0.88	100,000	7,557	515	99,805	195	0.19%

Script	Time (s)	Mem (MB)	CPU (%)	Read (MB)	Write (MB)
E01_sample_slimpajama	18.63	1252.86	42.9	524.6	424.34
E02_embed	154.05	1880.97	155.66	879.41	152.29
E03_hnsw_eval	4.0	1825.36	175.38	1.1	0.0
E04_hnsw_sweep	13.0	6287.91	151.11	0.0	0.0
E05_make_candidates	9.01	526.67	1354.08	0.01	0.0
E06_semdedup_hnsw	3.0	1227.0	101.44	0.01	0.0
E07_threshold_sweep	2.0	550.7	102.5	0.0	0.0
E08_build_dedup_dataset	2.0	144.97	102.17	0.37	199.95
Total	205.7	13696.45	273.15	1405.5	776.58



為評估 SemDedup 的去重效果撰寫了分析程式，比較原始資料集中高相似度文件對與 SemDedup 所識別出的重複文件對。我們先從原始資料集中擷取相似度高於指定門檻值的文件對，再檢查 SemDedup 是否成功識別並刪除這些重複文件。

實驗結果顯示，SemDedup 具有相當優異的重複文件偵測能力。在所有測試設定下，其所識別之重複文件對與 Ground Truth 高度一致。特別是在資料規模提升至 50,000 筆文件，且相似度門檻設定為 0.9 的情況下，SemDedup 仍成功找出所有高相似度文件對，達到 100% 的 Pair Recall。

```
=====
Dedup Recall Evaluation (EzSemDedup + HNSW)
=====
Loading embeddings...
Dataset size: 50000

Building ground truth pairs...
DEBUG: 目前這批資料的最大相似度是 0.9933
GT pairs: 81 (took 90.59s)

Loading predicted pairs...
Pred pairs: 81 (took 0.01s)

Computing pair recall...
Pair Recall: 1.000000
TP: 81, FN: 0

Computing cluster recall...
Cluster Recall: 1.000000

Compression:
Drop Rate (approx): 0.9998

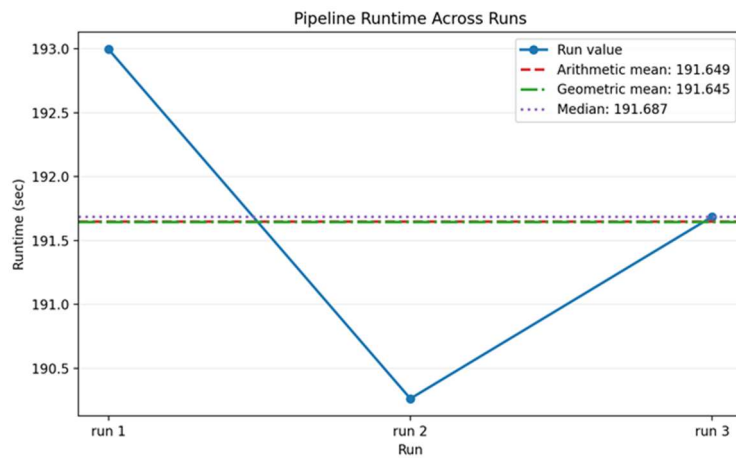
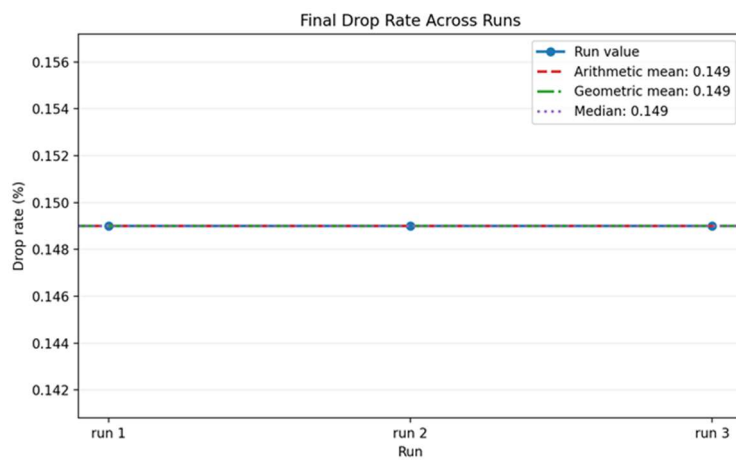
Computing End-to-End SemDedup recall...
Surviving documents (in top 50000): 49,926 (took 0.99s)
End-to-End Recall: 1.000000
Successfully deduplicated (TP): 81, Leaks (FN): 0
```

V4 : Optimized HNSW-based SemDeDup

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 191.6867
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_hnsw_ImproveSemDedup/pipeline_20260614_005409.log
=====
```

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 193.8038
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_hnsw_ImproveSemDedup/pipeline_20260614_031520.log
SUMMARY SAVED TO: /home/u08/workspace/HNSW/data/reports/pipeline_summary_20260614_031520.json
=====
```

```
=====
PIPELINE FINISHED
TOTAL TIME SEC: 195.6046
LOG SAVED TO: /home/u08/workspace/HNSW/log/slim_hnsw_ImproveSemDedup/pipeline_20260614_033432.log
SUMMARY SAVED TO: /home/u08/workspace/HNSW/data/reports/pipeline_summary_20260614_033432.json
=====
```



四、結論

在我們的實驗中首先討論 SlimPajama 作為資料集的可能和適配的模式，並根據經驗法則挑選適合模型的參數；接著以此為基礎，探討維持去重能力的情況下是否能夠降低計算成本，完成輕量有效的去重。

從前面實驗一~實驗三中可以知道，SlimPajama 作為有基礎前置處理的大型語言資料集，依舊還是有語意模糊的空間可讓我們改善，且單純透過 transformer 處理依舊會有候選對品質的問題，因此我們設計了兩種改良版本

1. HNSW + EzSemDeDup：保留 SemDeDup 原有 K-means 分群流程。
2. HNSW + ImproveSemDeDup：移除 SemDeDup 中的 K-means 分群步驟。

測試結合了 HNSW 後是否能

1. 提升 SemDedup 選擇候選對的能力
2. 減少 SemDedup 一開始做 K-Means 的運算壓力

並且同時以不掉 Recall 為目的，實驗結果整理成以下表格。

指標	HNSW	SemDeDup	HNSW + EZSemDeDup	HNSW + ImproveSemDeDup
執行時長(s)	183	951	205	191
Drop Rate(%)	-	0.189	0.195	0.149
時間換算	1	-	1.120	1.044
時間換算	-	1	0.216	0.201
drop rate換算	-	1	1.032	0.784

在 10,000 筆 SlimPajama 子資料集上的實驗結果顯示，原始 SemDeDup 需耗時 951 秒，而 HNSW + EzSemDeDup 僅需 205 秒，執行時間降低至原方法的 21.6%；HNSW + ImproveSemDeDup 則進一步縮短至 191 秒，僅為原方法的 20.1%。

在去重效果方面，SemDeDup 的 Drop Rate 為 0.189%，HNSW + EzSemDeDup 為 0.195%，兩者差距僅約 3.2%，顯示 EzSemDeDup 幾乎完整保留了原始 SemDeDup 的去重能力。相較之下，HNSW + ImproveSemDeDup 的 Drop Rate 為 0.149%，約為 SemDeDup 的 78.4%，雖然執行時間略有改善，但犧牲了部分去重效果。

根據研究初期所設定之目標：

- (1) 相較於純 HNSW 流程，額外執行時間增加不超過 5%；
- (2) 相較於 SemDeDup，Drop Rate 損失不超過 5%。

目前結果顯示，HNSW + EzSemDeDup 的執行時間較 HNSW 增加約 12%，未完全達成時間目標，但其 Drop Rate 與 SemDeDup 幾乎相同，成功達成品質目標。另一方面，HNSW + ImproveSemDeDup 的執行時間較 HNSW 僅增加約 4.4%，符合時間目標，但其 Drop Rate 下降約 21.6%，未能維持與 SemDeDup 相近的去重效果。

因此，若以目前實驗結果評估，HNSW + EzSemDeDup 在效率與品質之間取得較佳平衡，可視為現階段較具潛力的方案。

五、未來展望

本研究成功將 HNSW 與 SemDeDup 整合，建立一套兼顧效率與語意去重能力的文件去重流程。實驗結果顯示，透過 HNSW 進行候選文件搜尋後，可大幅降低 SemDeDup 的計算成本，使整體執行時間由 951 秒縮短至約 200 秒左右，達到接近 80% 的時間節省。

在兩種改良版本中，HNSW + EzSemDeDup 能夠維持與原始 SemDeDup 幾乎相同的 Drop Rate，同時大幅減少執行時間，顯示 HNSW 所產生的候選集合已足以覆蓋大部分語意重複文件。另一方面，移除 K-means 的 ImproveSemDeDup 雖進一步降低執行成本，但也造成去重能力下降，顯示分群機制在 SemDeDup 中仍具有一定的重要性。

此外，我們透過額外設計的 Pair Recall 驗證流程，確認 SemDeDup 所識別之高相似度文件對與 Ground Truth 高度一致。在部分設定下甚至達到 100% Pair Recall，進一步驗證本研究方法在維持去重品質方面的可行性。

目前實驗僅在 10,000 筆 SlimPajama 子資料集上進行驗證，因此未來將進一步擴展至更大規模資料集（例如 50K、100K 甚至更大規模），觀察不同方法在資料量增加時的可擴展性（Scalability）與穩定性。

此外，本研究後續將著重於以下幾個方向：

- 大規模資料驗證：比較 HNSW + EzSemDeDup 與 HNSW + ImproveSemDeDup 在不同資料規模下的執行時間、記憶體消耗與 Drop Rate 表現，以確認最適合實際應用的架構。
- 多來源 Candidate Generation：目前候選文件主要來自單一 embedding 模型，未來可結合多種 embedding 模型或不同相似度量測方式，以提升候選集合的召回率。
- 完整品質評估：除 Drop Rate 外，未來將進一步分析 Pair Recall、Cluster Recall、Precision 等指標，以更全面評估語意去重系統的實際效能。

透過上述研究方向，期望能建立一套兼具高效率、高召回率以及良好擴展性的語意去重系統，適用於大型語言模型訓練資料之前處理流程。